P2509R0
A proposal for a type trait to detect value-preserving conversions

Published Proposal, 2021-12-15

Issue Tracking:
Inline In Spec

Author:
Giuseppe D'Angelo

Audience:
SG6, LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to add a type trait to detect conversions between arithmetic-like types that always preserve the numeric value of the source object.

Table of Contents

§ 1. Changelog

- R0
  - First submission.

§ 2. Motivation and Scope

Consider a datatype built on top of an arithmetic type:

```
template <class Rep> // an arithmetic-like type
class Quantity
{
    Rep value;

    ~~~
};
```

Examples of such a datatype include the class templates `std::complex`, `std::chrono::duration`, [QAngle] (proposed for inclusion in Qt), and so on.

It’s reasonable to expect that different specializations of this datatype may convert to each other: a user may want to convert a `Quantity<int>` to, say, a `Quantity<long>`.

This is typically realized via a *converting constructor*:

```
template <class Rep>
class Quantity
{
    Rep value;

public:
    template <class Rep2>
    /* explicit/requires (some expression depending on Rep and Rep2) */
    Quantity(const Quantity<Rep2>& other);

    ~~~
};
```

Such a converting constructor is usually somehow "constrained" in order to prevent information loss and/or undefined behavior from occurring, as shown in the snippet above. Examples include:

1. the converting constructor from `std::complex<From>` to `std::complex<To>` is `explicit` if and only if the conversion from `From` to `To` is a narrowing conversion ([dcl.init.list], [complex.special]). Although the Standard does not use the "narrowing conversion" wording (instead, it fully specifies each and every specialization, marking the converting constructors `explicit` as appropriate in each specialization), and [P1467R7] changes the related wording, the semantics are indeed matching the ones of detecting a narrowing conversion (see also [P0870R4]'s motivating examples);
2. similarly, the converting constructor from `QAngle<From>` to `QAngle<To>` is `explicit` if and only if `To` cannot exactly represent any possible value of `From`. For instance, this implies that `QAngle<int>` to `QAngle<double>` is implicit on x86-64 using the Itanium ABI (as any 32 bit `int` can be precisely converted to a `binary64` `double`);
3. `std::chrono::duration` uses some ad-hoc detections to exclude its converting constructor from overload resolution ([time.duration.cons]). These include handling of the tick period, as well as the ad-hoc trait `std::chrono::treat_as_floating_point`.

Each approach has its own advantages and disadvantages.

1. "Constraining" on narrowing conversions allows one to simply side-step the problem of defining the semantics involved -- one can simply refer to the core language definition of narrowing. On

the other hand, this may yield counter-intuitive results. For instance a conversion between `int` and `long double` is always considered narrowing, even if it would precisely preserve the source values, and therefore make `Quantity<int>` not implictly convert to `Quantity<long double>`. This could be surprising for users.

2. "Constraining" on whether the conversion would precisely preserve any possible source value may give more expected results, in line with the idea that implicit conversions never lose information. On the other hand, this would risk limiting the portability of the one's source code, by introducing source incompatibilities when porting the code to a new platform. For instance, this could mean that `Quantity<long double>` would be implictly convertible to `Quantity<double>` on x86-64 when using the MSVC ABI (where both `long double` and `double` actually use `binary64`, and therefore can represent the very same values), but not on the Itanium ABI (where `long double` instead uses the x86 80-bit extended precision format).

3. An ad-hoc approach allows for maximum flexibility in terms of semantics, but it may also be surprising or frustrating to use correctly. For instance, a user defining a custom floating-point type (e.g. `float16`) may forget to also specialize `std::chrono::treat_as_floating_point`, therefore causing confusion when using something like `std::chrono::duration<float16>`:

```
using namespace std::chrono;
duration<double>  dd;
duration<float>   df   = dd;  // OK even if narrowing
duration<float16> df16 = dd;  // ERROR unless treat_as_floating_point h
```

The Standard Library does not provide any facilities for helping in the implemention of cases 1 and 2. (Case 3 is by definition ad-hoc and therefore one cannot provide generic facilities for it.)

Case 1 is supposed to be tackled by [P0870R4], which aims at introducing a type trait to detect narrowing conversions.

We are therefore left with case 2, which is the subject of the present proposal. Here we propose to add a type trait to detect whether a conversion exists between two types and that conversion always preserves values exactly.

§ 3. Impact On The Standard

This proposal is a pure library extension. It proposes changes to the `<type_traits>` header.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

[P0870R4] ("A proposal for a type trait to detect narrowing conversions") is very related to this proposal. During a mailing list review of [P0870R4] by SG6, it has been brought forward that users of that trait might find some behaviors counter-intuitive. The trait described by the present proposal complements [P0870R4]'s.

This proposal is related to [P1467R7] ("Extended floating-point types and standard names"), in at least two important aspects. First, the trait that we are proposing is going to defined in a way that correctly interoperates with the proposed extended floating-point types. Second, [P1467R7] introduces a *conversion rank* of floating-point types. This is done in order to properly extend the definition of usual arithmetic conversions to the extended floating-point types. Our proposal is however *not* going to make use of this ranking, for the simple reason that the ranking does not necessarily take into account the set of representable values of each floating-point type. For instance, `long double` is considered to unconditionally have higher ranking than `double`, even on architectures where the two types have an identical representation. This aspect, as well as the impact of [P1467R7] on the definition of narrowing conversions, is discussed in the § 4.4 Is the proposed trait different from simply detecting narrowing conversions? paragraph.

[P1619R1] and [P1998R1] introduce functions (called respectively `can_convert` and `is_value_lossless_convertable`) that check whether a given value of an integer type can be represented by another integer type. This is in line with the spirit of detecting value-preserving conversions, namely, preventing loss of information and/or preventing undefined behavior. While this proposal works on types, the functions examine specific values; we therefore think that the proposals are somehow orthogonal to the current proposal.

Finally, [P1841R1] ("Individually Specializable Numeric Traits") is proposing to add individual traits for numeric types, complementing and/or replacing the information that is currently found in the `numeric_limits` class template. We do not see this as a problem, as the functionality that we need in order to implement the trait we are proposing is found, with identical meaning, both in `numeric_limits` and in [P1841R1]'s proposed traits. However, this "double definition" could impact the desired wording. Here, we seek SG6 and LEWG guidance.

§ 4. Design Decisions

§ 4.1. What defines a "value-preserving conversion"?

Answering this question accurately is essential in order to have a proper definition for the type trait that we are proposing.

Given a conversion from type `From` to a type `To`, the semantic operation that we want to model is that the value represented by the `From` object preserves its numeric value after the conversion.

Giving an operational definition is challenging, due to how C++ implicit conversions operate.

Note that merely requiring that a "round-trip" conversion from `From`, to `To`, and back to `From` to yield back the original value is a necessary but not sufficient condition. (For instance, `int` to `unsigned int` is not a value-preserving conversion, despite the fact that all such round-trip conversions would keep the original values.) Similarly, requiring that the original value and the value after the conversion compare equal (incl. taking NaN into account) is necessary but not sufficient.

Instead, one can give a semantic definition: a conversion is value-preserving if and only if any possible value of type `From`, when converted to type `To`, is exactly represented by the result of the conversion. This includes numeric values, but also special values (NaN, infinities, signed zeroes, ...) in case of floating-point types. The wording "exactly represented" is already used by core language when dealing with conversions (cf. [conv.double], [conv.fpint]), so we don't have to define it ourselves.

The above semantics can be expressed in generic code by using the facilities provided by `numeric_limits`.

(For instance, an unsigned integer type `I1` has a value-preserving conversion towards a signed integer type `I2` if and only if `numeric_limits<I1>::radix` raised to the power of `numeric_limits<I1>::digits` is less than or equal to `numeric_limits<I2>::radix` raised to the power of `numeric_limits<I2>::digits`. On the other hand, there is no value-preserving conversion from `I2` to `I1`.)

This can be generalized to all the other conversions between arithmetic types, by using their signedness (`is_signed`), the `radix`, the number of digits (`digits`), and for floating-point numbers (`is_integral` is `false`) the `max_exponent` (or equivalently the minimum and maximum finite values representable, by using `min()` and `max()` respectively).

§ 4.2. What about unbounded types?

`numeric_limits` allows to identify types that may represent a non-finite set of values via the `is_bounded` static data member. While all foundamental types are bounded, a user may define unbounded arithmetic types (for instance, an arbitrary precision type).

The issue with such types is that we cannot entirely reason about them in terms of `numeric_limits` data members / member functions: many of them are not meaningful for unbounded types.

We can universally claim that the following conversions are not value-preserving:

- from an unbounded type to a bounded type;
- from a signed integer type to an unsigned integer type (bounded or not);
- from a floating-point type to an integer type (bounded or not);
- from a floating-point type that can represent special values (infinities, NaN, etc.) to a floating-point type (bounded or not) that cannot represent those values.

We cannot however reason about are the conversions:

- from an unsigned integer type towards an unbounded signed integer type;
- from an arbitrary type towards an unbounded floating-point type,

because there is no way (in general) to know what is the set of representable values of an unbounded type. For instance, an implementation of a unbounded type may use a "default" precision but still let users tune it at runtime, globally and/or on a per-object basis. A conversion towards an object of "default" precision may cause information loss; while tuning the precision and then doing the conversion would not. Since this property is not a static property of the `To` type, we cannot statically reason about it.

We are therefore going to make a judgement call for these last two cases: if it exists an implicit conversion between a type `From` and an unbonded type `To`, and we cannot otherwise establish that the conversion is not value-preserving, then we are going to assume that the conversion *is* value-preserving (in other words, that the `To` type is always going to use enough precision to correctly represent any possible value of `From`).

§ 4.3. Should the type trait be specializable for user-defined datatypes?

For the moment, we are *not* proposing it. This is consistent with the other type traits defined in [meta] (cf. [meta.rqmts]/4).

However, a program may add specializations to `numeric_limits` (or equivalently to [P1841R1]'s traits) for user-defined datatypes. We expect the type trait that we are proposing to be indeed defined in terms of `numeric_limits`, and therefore we believe that users can use that customization point in order to properly define the behavior of our type trait. For this very reason, we are also *not* limiting our type trait to work only with fundamental types.

§ 4.4. Is the proposed trait different from simply detecting narrowing conversions?

It is, in several ways:

- The definition of narrowing conversions between floating-point types do not take into account the actual values representable by them. For instance, a conversion from `long double` to `double` is always considered narrowing ([dcl.init.list]/7.2) even on architectures where the two datatypes are backed by the very same representation (for instance on architectures using IEEE754's `binary64` representation, such as x86-64 under the MSVC ABI). The trait we are proposing would instead take into account implementation-specific quantities. [P1467R7] proposes changes to the definition of narrowing conversions between floating-point types (by introducing conversion ranks); such changes are in line with the existing definition, in the sense that they do not necessarily take into account the actual values representable by a given type.
- Conversions from integer types to floating-point types are always considered narrowing, even when the destination type can precisely represent all the values of the source type ([dcl.init.list]/7.3). This is the case for instance between `int` and `double` on x86-64 (under all the commonly used ABIs).
- Conversions from pointers to `bool` are considered to be narrowing ([dcl.init.list]/7.5); however, the trait we are proposing only deals with arithmetic-like types.
- Similarly, we are not dealing with (unscoped) enumeration types, which are not arithmetic-like types.

For these reasons we believe the trait we are proposing is actually complementing [P0870R4]'s `is_convertible_without_narrowing`, thus giving users the ability of choosing the trait that best serves their use cases.

§ 4.5. When should users detect narrowing conversions vs. value-preserving conversions?

It is hard to give a clear-cut answer to this question. The Standard Library itself is inconsistent in this regard.

We believe that each option comes with pros and cons (e.g. flexibility vs. portability, see § 2 Motivation and Scope) that each user has to evaluate for themselves. We believe however that it is important to offer *both* options so that users *can* make the choice.

§ 4.6. Bikeshedding: naming

Many thanks go to Matthias Kretz, who proposed `is_value_preserving_conversion` on SG6's reflector. We've just adapted the name to make it more in line with the other existing traits.

§ 5. Implementation experience

A working prototype of the changes proposed by this paper, done on top of GCC 11, is available in this GCC branch on GitHub.

§ 6. Technical Specifications

All the proposed changes are relative to [N4892].

§ 6.1. Feature testing macro

Add to the list in [version.syn]:

```
#define __cpp_lib_is_value_preserving_convertible YYYYMML  // also in <type
```

with the value specified as usual (year and month of adoption).

§ 6.2. Proposed wording

Modify [meta.type.synop] as follows:

```
template<class From, class To> struct is_nothrow_convertible;
template<class From, class To> struct is_value_preserving_convertible;
```

```
template<class From, class To>
  inline constexpr bool is_nothrow_convertible_v = is_nothrow_convertible<F
template<class From, class To>
    inline constexpr bool is_value_preserving_convertible_v = is_value_pres
```

Add a new row to the "Type relationship predicates" ([tab:meta.rel]) table:

> **Template**: `template<class From, class To> struct is_value_preserving_convertible;`
>
> **Condition**: *see below*
>
> **Comments**: `numeric_limits<From>::is_specialized` shall be `true`, and `numeric_limits<To>::is_specialized` shall be `true`.

ISSUE 1 what about [P1841R1]'s traits?

ISSUE 2 is L(E)WG fine at adding a dependency from `<type_traits>` to `<limits>`?

Add a new paragraph at the end of [meta.rel]:

> **6** The predicate condition for a template specialization `is_value_preserving_convertible<From, To>` is satisfied if and only if `is_convertible_v<From, To>` is `true`, and each and every possible value representable by a source object of type `From` is exactly represented by the object obtained after converting the source object from `From` to `To` using an implicit conversion ([conv]). [*Note 4*: This includes values such as infinity, quiet and signaling "Not a Number", and so on. -- *end note*] [*Note 5*: If `numeric_limits<From>::is_bounded` is `false` and `numeric_limits<To>::is_bounded` is `true`, then the predicate condition shall not be satisfied. If `numeric_limits<To>::is_bounded` is `false`, an implementation is allowed to assume that any value representable by a source object of type `From` is exactly represented by the object obtained after converting the source object from `From` to `To`, unless it can otherwise detect that this is not the case (for instance, if `From` is a signed integer type and `To` is unsigned). —*end note*]

§ 7. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[N4892]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4892.pdf

[P0870R4]

Giuseppe D'Angelo. *A proposal for a type trait to detect narrowing conversions*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0870r4.html

[P1467R7]

David Olsen; Ilya Burylov; Michał Dominiak. *Extended floating-point types and standard names*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1467r7.html

[P1619R1]

Lisa Lippincott. *Functions for Testing Boundary Conditions on Integer Operations*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1619r1.pdf

[P1841R1]

Walter E. Brown. *Individually Specializable Numeric Traits*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1841r1.pdf

[P1998R1]

Ryan McDougall. *Simple Facility for Lossless Integer Conversion*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1998r1.pdf

[P2509-GCC]

Giuseppe D'Angelo. *P2509 prototype implementation*. URL: https://github.com/dangelog/gcc/tree/std-proposals

[QAngle]

Giuseppe D'Angelo. *Long live Q(Generic)Angle!*. URL: https://codereview.qt-project.org/c/qt/qtbase/+/191717

§ Issues Index

ISSUE 1 what about [P1841R1]'s traits? ↲

ISSUE 2 is L(E)WG fine at adding a dependency from `<type_traits>` to `<limits>`? ↲